Week 9: Cloud Data with Supabase

Python 200 | Mentor Session

What This Week Covers

Students move from concepts to code. The local scripts from weeks 1–7 stored output on disk. This week, data lives in a cloud database.

- **Supabase setup** — creating a project, locating credentials
- **Credential management** — `.env` files and `python-dotenv`
- `supabase-py` — connecting to a hosted Postgres database from Python
- **The two-table schema** — `weather_raw` and `weather_enriched`
- **CRUD operations** — insert, select, upsert, delete
- **Extract + Load pipeline** — fetch a year of weather data from Open-Meteo, load it into `weather_raw`

By the end of the week, students have 365 rows of real weather data in a cloud database, ready for the ML and LLM transforms in weeks 10–11.

What Supabase Is

Supabase is a hosted PostgreSQL database with a Python SDK, a web dashboard, and an auto-generated REST API.

Unlike file-based cloud storage (S3, Azure Blob), Supabase stores **rows and columns**.

With a table, you can:

- Query by field value: `WHERE date = '2023-06-15'`
- Filter by range: `WHERE temperature_2m_max > 30`
- Count records, check for gaps, join tables

None of that is possible with a folder of files.

> **Discussion:** A student says "why not just save the API response as a JSON file in cloud storage?" What can't you do with the file that you can do with a table?

Credentials: The Right Way

API keys must never appear in source code. Committing a key to a public GitHub repo can expose it within minutes.

The standard pattern:

1. Create a `.env` file in your project directory:

```
SUPABASE_URL=https://<project-id>.supabase.co
SUPABASE_KEY=<your-anon-key>
```

2. Add `.env` to `.gitignore` — it never gets committed.

3. Load with `python-dotenv` in your script:

```
import os
from dotenv import load_dotenv

load_dotenv()
supabase_url = os.getenv("SUPABASE_URL")
supabase_key = os.getenv("SUPABASE_KEY")
```

This pattern applies to **every** API key, not just Supabase.

Connecting with supabase-py

```
from supabase import create_client

supabase = create_client(
    os.getenv("SUPABASE_URL"),
    os.getenv("SUPABASE_KEY"),
)
```

The `supabase` object is the entry point for all database operations. Every operation follows the same pattern:

```
response = supabase.table("weather_raw").select("*").execute()
rows = response.data   # list of dicts
```

All operations end with `.execute()`. The result is always in `response.data`.

> **Discussion:** `os.getenv("SUPABASE_URL")` returns `None` and the script crashes. What are the two most likely causes?

Lesson 1 Demo: Connecting to Supabase

Open: Lesson 1 → connecting to Supabase

Focus on	Skim for now
The `.env` pattern — why it exists, how `load_dotenv()` works	Supabase dashboard navigation details
`create_client()` — one line, one object	Row Level Security concepts (covered in Lesson 2)
`response.data` — always a list of dicts	`supabase-py` version history

Walk through the connection verification step together: create a test table in the dashboard, run the select script, confirm the response.

> **Discussion:** A student asks "can I just hardcode the URL and key in my script since it's just for class?" What's the actual risk, and how would you respond?

The Two-Table Design

The pipeline needs two storage zones: raw data and enriched data. Keep them in **separate tables**.

`weather_raw` — exactly what the API returned. Immutable once loaded.
`weather_enriched` — ML predictions + LLM recommendations. Written by the transform step.

Why separate tables?

- **Raw data is immutable.** If the enrichment step has a bug, re-run it against the original data — it's still there, unchanged.
- **Each stage is independently queryable.** "Which dates are in `weather_raw` but not yet enriched?" is one SQL query.
- **Mirrors the pipeline stages.** Extract writes to `weather_raw`. Transform reads from it and writes to `weather_enriched`.

Creating the Tables

Run in the Supabase SQL Editor (`</>` in the sidebar):

```
CREATE TABLE weather_raw (
  date                 date        PRIMARY KEY,
  temperature_2m_max   numeric,
  temperature_2m_min   numeric,
  precipitation_sum    numeric,
  wind_speed_10m_max   numeric,
  loaded_at            timestamptz DEFAULT now()
);

CREATE TABLE weather_enriched (
  date              date        PRIMARY KEY REFERENCES weather_raw(date),
  good_for_running  boolean,
  confidence        numeric,
  llm_summary       text,
  enriched_at       timestamptz DEFAULT now()
);
```

Then disable Row Level Security on both tables so scripts can read and write freely.

Two Design Details Worth Discussing

`date` as primary key — weather data is one record per day, so the date is a natural unique identifier. No auto-increment ID needed.

Foreign key on `weather_enriched.date` — this constraint enforces pipeline ordering at the database level: you can only enrich a date that already exists in `weather_raw`. If the extract step hasn't run, the enrichment insert fails with a clear error.

Column names match the API — `temperature_2m_max` is both the Open-Meteo field name and the Supabase column name. This is intentional: no renaming is needed when passing rows to the ML classifier later.

> **Discussion:** Why is it useful to have the database reject an enrichment record for a date that doesn't exist in `weather_raw`, rather than letting the script handle that check?

CRUD with supabase-py

All operations use the same fluent style:

```
# Insert
supabase.table("weather_raw").insert(record).execute()

# Select all
response = supabase.table("weather_raw").select("*").execute()
rows = response.data

# Filter
response = supabase.table("weather_raw").select("*").eq("date", "2023-01-15").execute()

# Upsert (insert or update on conflict)
supabase.table("weather_raw").upsert(records, on_conflict="date").execute()

# Delete (always use a filter)
supabase.table("weather_raw").delete().eq("date", "2023-01-01").execute()
```

Rule: use `upsert` instead of `insert` for pipeline load steps. More on this in a moment.

Lesson 2 Demo: Working with Tables

Open: Lesson 2 → supabase tables

Focus on	Skim for now
Two-table design — why raw and enriched are separate	RLS policy syntax
The SQL to create both tables — primary key, foreign key, `DEFAULT now()`	`.gte()`, `.lte()`, other filter methods
Upsert vs insert — when and why to use each	The complete CRUD script at the end of the lesson

Run the complete CRUD script together. Insert a record, read it back, upsert a change, confirm the update, delete it.

> **Discussion:** `supabase.table("weather_raw").delete().execute()` — no filter. What happens? Why is this especially dangerous in a pipeline?

The Extract + Load Pipeline

```
# Extract: fetch from Open-Meteo API
response = requests.get(url, params=params)
response.raise_for_status()
daily = response.json()["daily"]

# Transform: columnar → row format
records = [
    {
        "date":                daily["time"][i],
        "temperature_2m_max": daily["temperature_2m_max"][i],
        "temperature_2m_min": daily["temperature_2m_min"][i],
        "precipitation_sum":  daily["precipitation_sum"][i],
        "wind_speed_10m_max": daily["wind_speed_10m_max"][i],
    }
    for i in range(len(daily["time"]))
]

# Load: upsert into weather_raw
supabase.table("weather_raw").upsert(records, on_conflict="date").execute()
```

The transform here is minimal — just reshaping the API response. The big transforms come in weeks 10–11.

Idempotency via Upsert

A pipeline step is **idempotent** if running it multiple times produces the same result as running it once.

`insert` on an existing primary key → error

`upsert` with `on_conflict="date"` → updates in place, no error, same final state

Why this matters:

- Pipelines get re-run: after a bug fix, after a partial failure, after an infrastructure outage
- Re-running an idempotent load step is always safe — no duplicates, no manual cleanup
- The 365 rows in `weather_raw` are the same whether you've run the script once or ten times

> **Discussion:** The pipeline crashes halfway through the LLM enrichment step. You fix the bug and re-run. What does idempotency buy you here — for both the load step and the transform step?

Lesson 3 Demo: Extract + Load Pipeline

Open: Lesson 3 → loading pipeline

Focus on	Skim for now
`raise_for_status()` — why it's called on every API response	Open-Meteo parameter details
Columnar → row format transformation	Handling `None` values for missing weather data
Upsert with `on_conflict` — idempotent by design	`count="exact"` verification query

Run the complete extract + load script. Confirm 365 rows in the Supabase Table Editor.

> **Discussion:** What would happen if you used `insert` instead of `upsert` and ran the script twice? What error would you see, and at what step?

Week 9 Wrap-Up

- **Supabase** — hosted Postgres; rows and columns, not files; queryable at every stage
- **Credentials** — always use `.env` + `.gitignore`; never hardcode secrets
- **`supabase-py`** — fluent API: every operation ends with `.execute()`; data is in `response.data`
- **Two-table schema** — `weather_raw` for immutable API data; `weather_enriched` for ML + LLM output
- **Foreign key** — enforces pipeline ordering at the database level
- **Upsert** — insert or update on conflict; makes load steps idempotent and safe to re-run

Takeaway: Moving data storage to the cloud changes where data lives, not how you reason about it. The 365 rows now in `weather_raw` are the starting point for everything in weeks 10–11: the ML classifier runs against them, the LLM enriches the results, and Prefect orchestrates the whole thing.